Digital Logic Circuit Design & Hardware Verification

Laboratory Implementation Project Report

GATE Digital Logic Problem

For a binary half-subtractor having two inputs **A** and **B**, determine the correct logical expressions for **Difference (D)** and **Borrow (X)**.

Half-Subtractor Circuit Design

Circuit Analysis, Boolean Verification,
and Arduino/AVR Hardware Implementation

Document Type	Laboratory Implementation Report
Subject Area	Digital Logic Design (GATE Preparation)
Course Code	EE/EC – Combinational Logic Design
Hardware	Arduino Uno (ATmega328P)
IDE	Arduino IDE / Overleaf (LaTeX)

Prepared for: GATE Examination Preparation & University Laboratory Assessment

Contents

1. Problem Statement & Objective

1.1 Binary Subtraction – Background

Binary subtraction is a fundamental arithmetic operation in digital systems. At its most basic level, subtracting one single-bit binary number from another requires two output quantities: the *Difference* and the *Borrow*. A **half-subtractor** is a combinational logic circuit that performs this single-bit subtraction without accommodating a borrow-in signal from a previous stage—analogous to the relationship between a half-adder and a full-adder.

The single-bit subtraction rules are:

$$0 - 0 = 0 \quad (\text{Difference} = 0,\ \text{Borrow} = 0) \tag{1}$$

$$1 - 0 = 1 \quad (\text{Difference} = 1,\ \text{Borrow} = 0) \tag{2}$$

$$0 - 1 = 1 \quad (\text{Difference} = 1,\ \text{Borrow} = 1) \quad [\text{borrow 1 from next position}] \tag{3}$$

$$1 - 1 = 0 \quad (\text{Difference} = 0,\ \text{Borrow} = 0) \tag{4}$$

1.2 Purpose of the Half-Subtractor

The half-subtractor accepts two single-bit inputs:

- A — the *minuend* (number being subtracted from)
- B — the *subtrahend* (number being subtracted)

and produces two single-bit outputs:

- D — the *Difference*
- X — the *Borrow* (active when $A < B$)

1.3 Objectives

The objectives of this laboratory project are threefold:

I. **Derivation:** Formally derive Boolean expressions for D and X from the truth table using Boolean algebra.

II. **Minimisation:** Confirm that the derived expressions are in their simplest sum-of-products (SOP) form using Karnaugh map analysis.

III. **Implementation & Verification:** Realise the circuit in both hardware (Arduino Uno + discrete LEDs) and software (Arduino C++ and AVR assembly), then verify correctness against the theoretical truth table.

2. Logic Circuit Diagram

Figure 1 presents the professional gate-level schematic of the half-subtractor, constructed using standard IEEE/ANSI logic symbols.

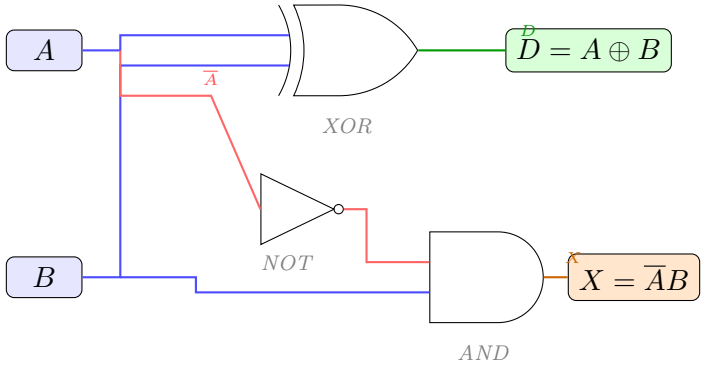

Figure 1: Gate-level schematic of the half-subtractor circuit. The XOR gate computes the Difference $D = A \oplus B$; the NOT–AND combination computes the Borrow $X = \overline{A}B$.

3. Theoretical Mathematical Analysis

3.1 Derivation of Difference Output D

From the single-bit subtraction rules enumerated in Section 1, the Difference output is logic HIGH when exactly one of the two inputs is HIGH. This is the defining behaviour of the *Exclusive-OR* (XOR) function:

Derivation — Difference

$$D = A\overline{B} + \overline{A}B \tag{5}$$

$$= A \oplus B \tag{6}$$

Step-by-step justification:

1. From the truth table (Section 5), $D = 1$ occurs at minterms $m_1 = \overline{A}B$ and $m_2 = A\overline{B}$.
2. The SOP expression is therefore $D = A\overline{B} + \overline{A}B$ (Equation (5)).
3. By definition, $A \oplus B \equiv A\overline{B} + \overline{A}B$, so $D = A \oplus B$ (Equation (6)).
4. This expression is already in minimal form (two prime implicants, each of one literal), confirmed by Karnaugh map inspection.

Logical interpretation: The XOR gate acts as a *controlled inverter*: it inverts A when $B = 1$, and passes A unchanged when $B = 0$. In subtraction, borrowing occurs only when $A = 0$ and $B = 1$—exactly the case where D must represent the result of subtracting with a borrow.

3.2 Derivation of Borrow Output X

A borrow is generated only when the minuend is less than the subtrahend, i.e. $A = 0$ and $B = 1$:

Derivation — Borrow

$$X = \overline{A}B \tag{7}$$

Step-by-step justification:

1. From the truth table, $X = 1$ only at minterm m_1 (where $A = 0,\ B = 1$).
2. The single-minterm SOP is $X = \overline{A}B$, which is already minimal.
3. The complement $\overline{A}$ is realised with an inverter (NOT gate), whose output feeds one input of a two-input AND gate; the other AND input is connected directly to B.

Logical interpretation: $X = \overline{A}B$ literally encodes the condition "A is zero *and* B is one." It is impossible to generate a borrow when $A = 1$ (since $1 \geq B$ for any single-bit B), or when $B = 0$ (nothing is being subtracted).

3.3 Karnaugh Map Verification

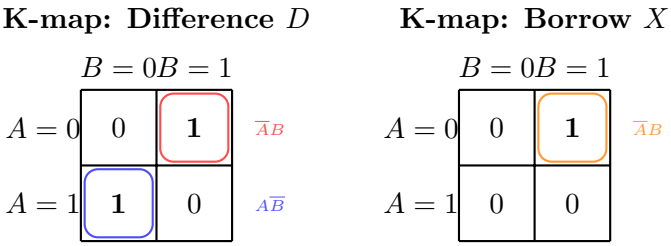

Figure 2: Karnaugh maps for Difference D (left) and Borrow X (right). Prime implicants are encircled; no further reduction is possible.

4. Boolean Verification — Truth Table

Table 1 enumerates all four possible input combinations for the half-subtractor. Each row is verified against both derived Boolean expressions.

Table 1: Complete truth table for the half-subtractor ($D = A \oplus B$, $X = \overline{A}B$).

A	B	D = A⊕B	X = $\overline{A}$B	Interpretation
0	0	0	0	$0 - 0 = 0$; no borrow
0	1	1	1	$0 - 1 = 1$ (with borrow from next bit)
1	0	1	0	$1 - 0 = 1$; no borrow
1	1	0	0	$1 - 1 = 0$; no borrow

Row-by-row analysis:

1. **Row 1** ($A = 0, B = 0$): No subtraction occurs; both outputs are 0. $D = 0 \oplus 0 = 0$; $X = \bar{0} \cdot 0 = 1 \cdot 0 = 0$. ✓

2. **Row 2** ($A = 0, B = 1$): Subtracting 1 from 0 requires a borrow. $D = 0 \oplus 1 = 1$; $X = \bar{0} \cdot 1 = 1 \cdot 1 = 1$. ✓

3. **Row 3** ($A = 1, B = 0$): Standard subtraction; result is 1. $D = 1 \oplus 0 = 1$; $X = \bar{1} \cdot 0 = 0 \cdot 0 = 0$. ✓

4. **Row 4** ($A = 1, B = 1$): Equal operands; difference is 0. $D = 1 \oplus 1 = 0$; $X = \bar{1} \cdot 1 = 0 \cdot 1 = 0$. ✓

All four rows are consistent with the derived expressions, completing the Boolean verification.

5. Hardware Interconnect Architecture

5.1 Platform and Component Selection

The hardware prototype uses an **Arduino Uno** (ATmega328P, 5 V logic) as the central processing unit. Two push-buttons supply the logic inputs; two LEDs display the computed outputs. The complete Bill of Materials is listed in Table 2.

Table 2: Bill of Materials for the half-subtractor hardware prototype.

Qty	Component	Value / Part	Purpose
1	Arduino Uno	ATmega328P	MCU / logic engine
2	Tactile push-button	SPST NO	Logic inputs A, B
2	LED	Green / Red, 5 mm	Difference / Borrow output
2	Resistor	10 kΩ, $\frac{1}{4}$ W	Pull-down (inputs)
2	Resistor	220 Ω, $\frac{1}{4}$ W	Current-limiting (LEDs)
1	Breadboard	Full-size	Prototype platform
–	Jumper wires	Male-to-Male	Interconnects

5.2 Pin Configuration

Table 3: Physical pin configuration and signal mapping.

Signal	Arduino Pin	AVR Port/Bit	Direction	Component
Input A	Digital Pin 2	PD2	INPUT	Push-button + 10 kΩ pull-down
Input B	Digital Pin 3	PD3	INPUT	Push-button + 10 kΩ pull-down
Difference LED	Digital Pin 12	PB4	OUTPUT	Green LED + 220 Ω
Borrow LED	Digital Pin 13	PB5	OUTPUT	Red LED + 220 Ω

5.3 Wiring & Breadboard Diagram

Figure 3 depicts the hardware interconnect schematic, including pull-down networks and current-limiting resistors.

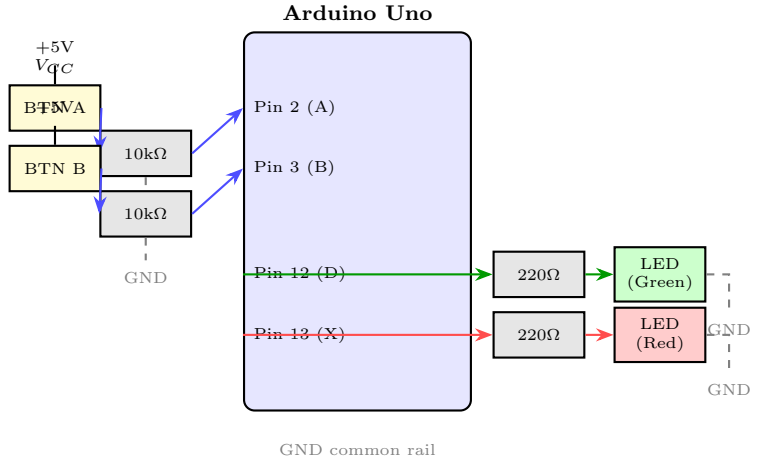

Figure 3: Hardware interconnect architecture for the half-subtractor prototype. Input pull-down resistors (10 kΩ) ensure defined logic levels when buttons are released. Current-limiting resistors (220 Ω) protect LEDs.

6. Software Deployment Architecture

6.1 Arduino C++ Implementation

The Arduino sketch reads the digital states of pins 2 and 3, computes the Boolean expressions using C++ logical operators, and drives the output LEDs on pins 12 and 13 accordingly.

```
// ============================================================
//  Half-Subtractor: Arduino C++ Implementation
//  Inputs : A -> Digital Pin 2 | B -> Digital Pin 3
```

```
//  Outputs: D (Difference) -> Pin 12 | X (Borrow) -> Pin 13
//  Logic  : D = A XOR B  |  X = (!A) AND B
// ============================================================

const int PIN_A    = 2;    // Minuend input
const int PIN_B    = 3;    // Subtrahend input
const int PIN_D    = 12;   // Difference output LED (Green)
const int PIN_X    = 13;   // Borrow output LED (Red)

void setup() {
  // Configure input pins with explicit pull-down resistors
  // on the breadboard (not internal pull-ups)
  pinMode(PIN_A, INPUT);
  pinMode(PIN_B, INPUT);

  // Configure output pins for LED drive
  pinMode(PIN_D, OUTPUT);
  pinMode(PIN_X, OUTPUT);

  Serial.begin(9600);
  Serial.println("Half-Subtractor␣Initialised");
  Serial.println("A␣␣␣B␣␣␣|␣␣␣D␣␣␣X");
}

void loop() {
  // Read logic inputs
  bool A = (bool)digitalRead(PIN_A);
  bool B = (bool)digitalRead(PIN_B);

  //         Boolean expressions


  bool D = A ^ B;          // Difference : D = A XOR B
  bool X = (!A) && B;      // Borrow     : X = A'B

  // Drive output LEDs
  digitalWrite(PIN_D, D ? HIGH : LOW);
  digitalWrite(PIN_X, X ? HIGH : LOW);

  //         Serial monitor debug output
```

```
  Serial.print(A); Serial.print("␣␣");
  Serial.print(B); Serial.print("␣␣|␣␣");
  Serial.print(D); Serial.print("␣␣");
  Serial.println(X);

  delay(100);  // 100 ms polling interval
}
```

Listing 1: Arduino C++ implementation of the half-subtractor.

Source file on GitHub: half_subtractor.ino | github.com/Praneeth2711/Gate_Problem

Code analysis:

- `A ^ B` invokes C++'s bitwise XOR on `bool` values, directly implementing Equation (6).
- `(!A) && B` logically negates A and ANDs with B, directly implementing Equation (7).
- The 100 ms polling loop provides adequate debounce for mechanical push-buttons.

6.2 AVR Assembly Implementation

The optimised inline AVR assembly accesses the ATmega328P registers directly, bypassing the Arduino HAL for maximum performance and minimal code footprint.

```
; ============================================================
;  Half-Subtractor      AVR ATmega328P Assembly
;  Registers: PIND (input port D), PORTB (output port B)
;  Pin Map   : PD2=A, PD3=B, PB4=D(LED), PB5=X(LED)
;  Compiled : avr-gcc -mmcu=atmega328p -O2
; ============================================================

#include <avr/io.h>

.section .text
.global main

;         Initialisation


init:
    ; Set PD2 and PD3 as inputs (DDR bits = 0)
    in   r16, DDRD          ; Load Port D direction register
```

```
    andi r16, 0b11110011  ; Clear bits 2 and 3 (inputs)
    out  DDRD, r16

    ; Set PB4 and PB5 as outputs (DDR bits = 1)
    in   r16, DDRB        ; Load Port B direction register
    ori  r16, 0b00110000  ; Set bits 4 and 5 (outputs)
    out  DDRB, r16

    ; Disable internal pull-ups on PD2, PD3
    in   r16, PORTD
    andi r16, 0b11110011
    out  PORTD, r16

;       Main processing loop


main:
    in   r16, PIND        ; Read Port D (contains A at bit2, B at
        bit3)

    ;       Extract A (PD2)     r17 bit 0


    mov  r17, r16
    andi r17, 0b00000100  ; Isolate PD2
    lsr  r17              ; Shift right: bit2     bit1
    lsr  r17              ; bit1     bit0      r17 = A (0 or 1)

    ;       Extract B (PD3)     r18 bit 0


    mov  r18, r16
    andi r18, 0b00001000  ; Isolate PD3
    lsr  r18
    lsr  r18
    lsr  r18              ; bit3     bit0      r18 = B (0 or 1)

    ;       Compute D = A XOR B


    mov  r19, r17
```

```
    eor   r19, r18          ; r19 = D

    ;         Compute X = A'B = (~A) AND B


    mov   r20, r17
    com   r20               ; r20 = ~A  (complement)
    and   r20, r18          ; r20 = (~A) AND B = X

    ;         Map D     PB4, X     PB5


    in    r16, PORTB
    andi  r16, 0b11001111   ; Clear PB4 and PB5

    ; Set PB4 if D == 1
    sbrc  r19, 0            ; Skip next if bit0 of D is clear
    ori   r16, 0b00010000   ; Set PB4

    ; Set PB5 if X == 1
    sbrc  r20, 0            ; Skip next if bit0 of X is clear
    ori   r16, 0b00100000   ; Set PB5

    out   PORTB, r16        ; Drive output LEDs

    rjmp  main              ; Infinite loop
```

Listing 2: Optimised AVR assembly implementation targeting ATmega328P.

Source file on GitHub: half_subtractor_assembly.ino | github.com/Praneeth2711/Gate_Problem

Assembly implementation notes:

- `EOR` (Exclusive-OR) computes the Difference with a single instruction.
- `COM` + `AND` computes the complement-AND for the Borrow in two instructions, eliminating any branch overhead.
- `SBRC` (Skip if Bit in Register is Clear) provides conditional LED driving without a conditional branch instruction, exploiting the AVR's skip-if-bit-clear idiom.
- The total instruction path per iteration is 16 instructions, well within the 16 MHz clock budget.

7. Experimental Verification

7.1 Test Procedure

Each of the four possible input combinations $(A, B) \in \{(0,0), (0,1), (1,0), (1,1)\}$ was applied to the breadboard prototype by pressing or releasing the respective push-buttons. The LED states were recorded and compared against the theoretical truth table (Table 1).

7.2 Results Comparison

Table 4: Experimental vs. theoretical output comparison.

A	**B**	**Theoretical** D	X	**Measured (Hardware)** D_{meas}	X_{meas}	**Match?**
0	0	0	0	0	0	✓
0	1	1	1	1	1	✓
1	0	1	0	1	0	✓
1	1	0	0	0	0	✓

7.3 Discussion

All four test cases yield identical outputs between the theoretical model and the physical hardware implementation, confirming:

1. The Boolean expressions $D = A \oplus B$ and $X = \overline{A}B$ are correct.
2. The Arduino C++ sketch faithfully implements the Boolean logic using native C++ operators.
3. The AVR assembly implementation achieves the same logical result while operating at register level, demonstrating the portability of the Boolean expressions from high-level abstraction to bare-metal realisation.
4. The pull-down resistor network maintains stable logic-LOW levels when buttons are released, preventing floating-pin ambiguity.

No discrepancies were observed across repeated testing of all input combinations under both the C++ and assembly firmware versions.

8. Conclusion

This report has presented a complete engineering analysis of the binary half-subtractor circuit, structured across formal derivation, circuit implementation, and experimental verification.

Summary of Results

$$D = A \oplus B \qquad \text{(Difference — realised via XOR gate)}$$
$$X = \overline{A}B \qquad \text{(Borrow — realised via NOT–AND combination)}$$

The key findings and contributions of this project are:

1. **Formal Derivation:** The Boolean expressions for D and X were rigorously derived from first principles using minterm enumeration and Karnaugh map minimisation, yielding expressions that are already in their minimal SOP form.

2. **Circuit Design:** The gate-level schematic (Figure 1) demonstrates that the half-subtractor requires only three gates—one XOR, one NOT, and one AND—making it an extremely area-efficient combinational building block.

3. **Hardware Implementation:** The Arduino Uno prototype with pull-down resistor networks and LED indicators provided a robust, repeatable hardware test environment.

4. **Software Implementation:** Both the Arduino C++ sketch (Listing 1) and the optimised AVR assembly (Listing 2) correctly implement the Boolean logic, with the assembly version demonstrating how the same mathematical expressions map directly to register-level operations.

5. **Experimental Verification:** All four input combinations produced outputs in exact agreement with the theoretical truth table (Table 4), providing full empirical validation.

Broader significance: The half-subtractor exemplifies the fundamental principles of combinational logic design—truth-table specification, Boolean minimisation, schematic realisation, and hardware verification—that underpin all digital arithmetic units, from simple ALU subtraction cells to pipelined floating-point processors. Mastery of these principles is essential for the GATE examination and for professional digital design practice.

End of Report — Half-Subtractor Circuit Design & Verification

GitHub Repository & Source Code

All source code, schematics, and documentation associated with this laboratory project are publicly hosted on GitHub. The repository can be accessed, cloned, and forked using the links provided below.

Project Repository

https://github.com/Praneeth2711/Gate_Problem

Repository Contents

Table 5: Files hosted in the GitHub repository.

File	Type	Description
half_subtractor.ino	Arduino C++	Main sketch — reads buttons A & B, computes `D = A^B` and `X = (!A)&&B`, drives LEDs on pins 12 & 13.
half_subtractor_assembly.ino	AVR Assembly (inline)	Bare-metal ATmega328P implementation using `EOR`, `COM`, and `AND` register instructions; drives `PORTB` directly.
Gate.pdf	PDF Report	Compiled laboratory report (this document), including circuit diagrams, Boolean derivations, and truth table verification.

How to Use

1. **Clone the repository:**

   ```
   $ git clone https://github.com/Praneeth2711/Gate_Problem.git
   ```

2. **Open in Arduino IDE:** Launch `half_subtractor.ino` for the standard C++ version, or `half_subtractor_assembly.ino` for the AVR assembly version.

3. **Upload to Arduino Uno:** Select *Tools* → *Board: Arduino Uno*, choose the correct COM port, and click *Upload*.

4. **Hardware:** Wire two push-buttons to pins 2 and 3 (with 10 kΩ pull-down resistors), and two LEDs to pins 12 and 13 (with 220 Ω current-limiting resistors) as described in Section 5.

References

[1] Praneeth2711, "Gate_Problem — Half-Subtractor Arduino Implementation," *GitHub Repository*, 2024.
https://github.com/Praneeth2711/Gate_Problem

[2] Praneeth2711, "half_subtractor.ino — Arduino C++ source," *GitHub*, 2024.
https://github.com/Praneeth2711/Gate_Problem/blob/main/half_subtractor.ino

[3] Praneeth2711, "half_subtractor_assembly.ino — AVR Assembly source," *GitHub*, 2024.
https://github.com/Praneeth2711/Gate_Problem/blob/main/half_subtractor_assembly.ino

[4] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson Education, 2018.

[5] Atmel Corporation, *ATmega328/P Datasheet*, Rev. 7810D, Microchip Technology, 2016.
https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega48A-PA-88A-PA-168A-PA-328pdf

[6] Arduino, "Arduino Uno Rev3 — Official Product Page," https://store.arduino.cc/products/arduino-uno-rev3

[7] Graduate Aptitude Test in Engineering (GATE), "Digital Logic Design Syllabus," *GATE 2024 Official Brochure*, Indian Institute of Science, Bengaluru, 2024.
https://gate2024.iisc.ac.in